

# ShinobuChat 智能桌宠聊天伴侣项目中期报告

来源文件：shinobu-chat-midterm-report.md

## 一、项目进展总览

### 1.1 项目基本信息

项目名称：ShinobuChat 智能桌宠聊天伴侣系统

项目定位：面向个人用户的桌宠式 AI 陪伴、对话、语音交互与轻量任务助手平台。

目标用户：

- 希望获得拟人化、持续陪伴式 AI 对话体验的个人用户。
- 希望通过桌宠、Live2D 角色、语音输入输出提升交互沉浸感的用户。
- 希望在聊天之外获得简单工具调用、信息整理、天气查询、待办总结等 Agent 能力的用户。
- 需要跨会话保留聊天历史、角色设置和桌宠偏好的长期使用者。

核心解决问题：

- 普通聊天机器人缺少持续陪伴感，界面和交互方式偏工具化。
- AI 回复等待过程缺少反馈，用户容易误以为系统卡死。
- 文字聊天无法满足语音输入、语音播报、情绪表达等多模态交互需求。
- Live2D、背景、音乐、角色偏好等资源通常难以灵活配置。
- Agent 工具能力与聊天体验割裂，用户无法在同一个界面中自然切换聊天、任务与工具调用。

核心价值：

ShinobuChat 将 AI 聊天、Live2D 桌宠、语音识别与合成、情绪反馈、会话记忆和技能调用整合到一个前后端分离的应用中，使用户能够在类似桌面伙伴的交互环境里完成自然聊天、语音沟通、轻量信息查询和任务辅助。

### 1.2 迭代计划执行情况

第 1-2 周：需求分析、功能边界确定、前后端分离架构设计、桌宠式交互方向确定。（已完成）

第 3-4 周：FastAPI 后端工程搭建、数据库模型设计、用户/会话/消息基础接口开发。（已完成）

第 5-6 周：React + Vite 前端工程搭建、三栏式主界面、聊天面板、会话列表和设置面板开发。（已完成）

第 7-8 周：AI 聊天接口接入、SSE

流式消息输出、前端流式渲染、会话持久化与状态合并逻辑开发。（已完成）

第 9 周：Live2D 资源管理、模型加载、情绪映射、背景与音乐资源清单机制开发。（已完成）

第 10 周：语音输入 ASR、语音合成 TTS、GPT-SoVITS

参考音频配置、情绪驱动语音/动作联动开发。（基本完成）

第 11-12

周：角色卡接口、权限收敛、编码问题修复、自动化测试补充、部署与演示稳定性优化。（进行中）

## 1.3 总体完成度

项目整体完成度：约 72%。

已完成模块：

- 后端 FastAPI 服务入口、路由注册、CORS、静态前端托管和健康检查。
- SQLAlchemy 数据模型，包括用户、设备、会话、消息。
- 用户注册、密码哈希、设备授权登录、JWT 生成。
- 会话列表、消息历史、消息创建与 SSE 流式回复。
- OpenAI 兼容 AI 接口配置，包括模型、Base URL、超时、token 上限等。
- Agent 模式、技能注册、技能匹配、工具调用框架。
- Live2D 模型清单、情绪映射、模型加载与前端舞台渲染。
- 前端三栏桌宠聊天界面、消息流式显示、会话切换、桌宠设置、音乐播放器。
- 语音输入与语音输出接口，支持 Whisper/FunASR ASR 和 GPT-SoVITS TTS。
- 前端部分单元测试，包括 SSE 解析、JWT 解析、消息状态合并、Live2D manifest 收集。

未完成或待完善模块：

- 角色卡前端已有调用入口，但后端 /characters/active 接口尚未实现。
- 部分中文文案存在编码异常，需要统一修复为 UTF-8。
- 当前后端多数接口仍以 user\_id 参数区分用户，尚未全面接入 JWT 鉴权依赖。
- 设备授权状态仍为内存存储，服务重启后会丢失。
- 后端自动化测试、数据库迁移、部署脚本和正式演示环境仍需补齐。

## 二、已完成功能详述

### 2.1 已完成功能列表

#### 2.1.1 后端服务体系

项目后端使用 FastAPI 构建，核心入口位于 `app/main.py`。系统在启动时初始化数据库表结构，统一挂载 `/api/v1` 业务接口，并在前端构建产物存在时直接托管 React 静态资源。

主要能力包括：

- 应用初始化与健康检查。
- API 版本化路由管理。
- CORS 跨域支持。
- 前端构建产物静态托管。
- 数据库启动初始化。
- 服务配置集中管理。

配置中心位于 `app/core/config.py`，通过 `pydantic-settings` 从 `.env` 读取环境变量，覆盖数据库地址、AI 模型、语音服务、ASR 引擎、JWT、资源目录等配置。

### 2.1.2 数据库与持久化

项目使用 SQLAlchemy ORM 管理数据模型，当前默认数据库连接为 PostgreSQL 协议，具备后续切换部署数据库的条件。

当前数据表包括：

- `users`：保存用户邮箱、密码哈希、昵称和创建时间。
- `devices`：保存设备码、设备名称、设备类型和最近活跃时间。
- `conversations`：保存用户会话标题、摘要、创建时间、更新时间。
- `messages`：保存会话内消息、角色、内容、路由模式和创建时间。

这些模型已经形成基础的长期记忆结构：用户的会话历史、消息记录和最近会话摘要可以跨页面刷新保留，也为后续角色记忆、长期偏好和主动服务打下基础。

### 2.1.3 用户注册与设备授权登录

系统已实现基础用户注册和设备授权登录流程。

用户注册流程：

用户提交邮箱、密码和昵称，后端检查邮箱是否重复，使用 `passlib` 对密码进行哈希存储，避免明文保存密码。

登录授权流程：

用户通过邮箱和密码发起设备授权，后端生成 `device_code` 和 `user_code`，随后设备使用 `device_code` 换取 JWT access token。前端从 token 中解析用户 ID，并把会话信息保存到 `localStorage`。

当前效果：

- 可以创建用户。
- 可以通过设备授权流程登录。
- 登录状态刷新后仍可恢复。
- 退出登录时清理本地会话、当前会话 ID 和消息状态。

#### 2.1.4 AI 聊天与 SSE 流式回复

消息服务位于 `app/services/message_service.py`，负责用户消息落库、会话创建、AI 请求、流式事件输出和助手消息保存。

流式事件包括：

- `conversation`：返回会话 ID、标题和用户消息。
- `chunk`：返回 AI 回复的增量文本。
- `progress`：返回 Agent/技能调用过程中的阶段状态。
- `emotion`：返回当前回复识别出的情绪。
- `done`：返回完整助手消息并标记流式回复结束。

前端使用 `fetch` 读取 `text/event-stream` 响应，并通过 `consumeSseStream` 逐段解析事件。消息列表会先显示本地 pending 消息，再在后端保存完成后替换为正式消息，从而避免刷新历史时丢失正在生成的回复。

实现效果：

- 用户发送消息后可以立即看到自己的消息入列。
- AI 回复逐段显示，不需要等待完整回复结束。
- 前端会根据 `done` 事件将临时消息替换为数据库保存后的正式消息。
- 会话标题会根据第一条用户消息自动生成。
- 会话摘要会根据助手回复自动更新。

#### 2.1.5 聊天、Agent 与自动路由模式

前端 `Composer` 提供三种路由模式：

- `chat`：普通聊天模式，适合自然对话和陪伴交流。
- `agent`：任务模式，优先调用技能和工具完成具体任务。
- `auto`：自动模式，为后续根据用户意图自动切换聊天与任务执行预留空间。

后端 Agent 服务具备以下能力：

- 从本地 `skills/*/SKILL.md` 读取技能说明。

- 根据用户输入匹配相关技能。
- 将技能说明注入 AI 上下文。
- 提供 `activate_skill`、`fetch_web_page`、`shell_command`、`conversation_digest` 等工具定义。
- 在流式响应中向前端输出工具执行进度。

当前已安装的本地技能包括天气、待办总结、晚间回顾、灵感整理、读屏和网页搜索聚合等，能够支撑后续从“聊天应用”扩展为“轻量 Agent 工作台”。

### 2.1.6 Live2D 桌宠与情绪联动

前端 Live2D 模块位于 `frontend/shinobu-chat/src/live2d/`，使用 `pixi.js` 和 `pixi-live2d-display` 加载 Cubism 4 模型。

已实现能力：

- 自动加载 Cubism Core 运行时。
- 从 `manifest` 读取 Live2D 模型、入口文件、缩略图、默认比例和情绪映射。
- 支持模型缩放、透明度、位置调整。
- 支持拖拽移动桌宠，滚轮调整缩放。
- 支持根据 AI 回复情绪触发表情或动作。
- 支持通过设置面板选择角色模型和背景。

当前资源清单中已有 Mao Pro EN 模型，并配置了 `neutral`、`happy`、`sad`、`angry`、`surprised`、`confused` 等情绪到表达/动作的映射。

### 2.1.7 语音输入与语音输出

语音服务位于 `app/services/voice_service.py`，前端通过 Composer 中的录音按钮采集浏览器音频。

语音输入：

- 前端使用 `MediaRecorder` 录制音频。
- 后端 `/api/v1/voice/asr` 接收上传文件。
- 支持 `whisper` 和 `funasr` 两种 ASR 引擎。
- 后端统一提取识别文本并返回 `{ text, engine }`。

语音输出：

- 前端在需要语音回复时调用 `/api/v1/voice/tts`。
- 后端根据文本和上下文识别情绪。
- 系统从 `voice_reference_audio.json` 或环境变量中选择对应情绪的参考音频。
- 后端请求 `GPT-SoVITS/tts` 接口，返回音频二进制。

- 前端创建 Blob URL 并播放助手语音。

该功能使 ShinobuChat 从纯文本聊天升级为可说、可听、可根据情绪选择语音参考的互动角色系统。

### 2.1.8 前端主界面与交互体验

前端使用 React 18、TypeScript、Vite 构建，主界面位于 frontend/shinobu-chat/src/App.tsx。

当前界面由三部分组成：

- 左侧导航栏：新对话、搜索、历史、角色模型、场景音乐和更多设置。
- 中央桌宠舞台：Live2D 角色、背景、互动工具、截图按钮和音乐播放器。
- 右侧聊天面板：状态栏、消息列表、输入框、路由模式切换、语音录制、Galgame 选项。

已实现的交互包括：

- 会话历史选择。
- 新建对话。
- 聊天消息自动滚动。
- 流式文本打字效果。
- 桌宠工具栏互动。
- 设置面板调整桌宠透明度、模型、背景和提醒方式。
- 音乐播放、上一首、下一首、循环和音量调节。
- 登录状态持久化和退出登录。

### 2.1.9 前端测试与工程化

前端已配置 Vitest，并已有以下测试：

- api/auth.test.ts：验证 JWT subject 解析逻辑。
- api/sse.test.ts：验证 SSE 原始事件解析和事件标准化。
- chat/messageState.test.ts：验证流式助手消息替换和本地消息保留。
- live2d/assetManifest.test.ts：验证 Live2D manifest 收集、默认配置合并和模型推断。

测试覆盖了当前最容易出现状态错乱的关键点：SSE

事件解析、聊天流式状态合并、模型资源清单处理和登录 token 解析。

## 2.2 核心功能演示

### 功能 1：登录后进入桌宠聊天工作区

用户输入：邮箱、密码、设备名称。

系统流程：

前端提交登录信息 -> 后端校验用户 -> 生成设备码 -> 前端用设备码换取 JWT -> 前端解析用户 ID -> 保存本地 session -> 进入 ShinobuChat 主界面。

运行效果：

用户登录后可以看到三栏式主界面，左侧为会话与功能导航，中间为 Live2D 桌宠舞台，右侧为聊天窗口。刷新页面后，前端可从 localStorage 恢复登录状态。

## 功能 2：AI 流式聊天与会话记忆

用户输入：任意聊天内容，例如“今天有点累，陪我聊一会儿”。

系统流程：

前端发送 /api/v1/messages 请求 -> 后端创建或复用会话 -> 保存用户消息 -> 组装历史上下文 -> 请求 AI 模型流式接口 -> 后端转发 chunk 事件 -> 前端实时渲染 -> 回复完成后后端保存助手消息 -> 前端替换临时消息。

运行效果：

AI 回复不再等待完整生成，而是边生成边展示。用户可以在历史会话中重新打开之前的聊天记录，系统也会维护会话标题和摘要。

## 功能 3：语音输入、语音回复与情绪驱动

用户输入：点击麦克风录音，说出一段语音。

系统流程：

浏览器录音 -> 上传音频 -> ASR 转文字 -> 将识别文本发送给消息接口 -> AI 生成回复 -> 情绪识别 -> 调用 TTS -> 返回音频 -> 前端播放语音并更新 Live2D 情绪。

运行效果：

用户可以使用语音向 Shinobu 提问，系统识别后自动进入聊天流程。助手回复完成后可合成语音播放，并根据 happy、sad、angry、thinking 等情绪改变角色表现。

## 2.3 智能特性实现情况

长期记忆：已实现基础版本。当前会话、消息历史和摘要已持久化到数据库，刷新页面或重新进入会话后仍可查看历史内容。

情绪识别：已实现规则版本。系统基于关键词对用户消息和助手回复进行 happy、sad、angry、surprised、thinking、neutral 分类，并用于语音参考选择和 Live2D 表情映射。

主动服务：部分具备基础。前端已有提醒方式设置、桌宠反馈和声音提示雏形，后续需要加入定时任务、用户偏好和主动推送策略。

Agent 技能：已实现初版。系统可以加载本地技能说明，并在 Agent 模式下使用工具框架完成网页获取、会话摘要等任务。

多模态交互：已实现初版。当前已有文字、语音输入、语音输出、Live2D 动作/表情和背景音乐等多模态元素。

## 三、技术实现与架构

### 3.1 实际技术架构

#### 3.1.1 前端技术栈

前端框架：React 18

语言：TypeScript

构建工具：Vite

UI 图标：lucide-react

Live2D 渲染：pixi.js、pixi-live2d-display

数据校验：zod

测试工具：Vitest、Testing Library、jsdom

核心能力：

- 三栏式桌宠聊天工作区。
- Live2D 模型加载与运行时管理。
- SSE 流式消息读取与标准化。
- 浏览器录音与音频播放。
- 资源 manifest 加载。
- localStorage 保存登录状态、会话 ID、路由模式和桌宠设置。

#### 3.1.2 后端技术栈

后端框架：FastAPI

语言：Python

数据库 ORM：SQLAlchemy



数据库驱动：psycopg

配置管理：pydantic-settings

认证安全：python-jose、passlib

文件上传：python-multipart

服务运行：uvicorn

核心能力：

- REST API 与 SSE 流式接口。
- 用户注册、密码哈希、JWT 生成。
- 设备授权登录。
- 会话和消息持久化。
- AI 聊天模型请求。
- Agent 工具调用。
- Live2D 资源 manifest 管理。
- ASR/TTS 语音服务代理。

### 3.1.3 整体数据流

flowchart LR

```
User["用户"] --> Frontend["React/Vite 前端"]
Frontend --> Auth["认证接口"]
Frontend --> MessageAPI["消息 SSE 接口"]
Frontend --> VoiceAPI["语音接口"]
Frontend --> Live2D["Live2D 舞台"]
```

```
Auth --> DB["PostgreSQL/SQLAlchemy"]
MessageAPI --> DB
MessageAPI --> AI["OpenAI 兼容聊天模型"]
MessageAPI --> Agent["Agent/Skill 服务"]
Agent --> Skills["本地 SKILL.md"]
VoiceAPI --> ASR["Whisper/FunASR"]
VoiceAPI --> TTS["GPT-SoVITS"]
```

```
AI --> MessageAPI
MessageAPI --> Frontend
TTS --> Frontend
Frontend --> User
```

## 3.2 关键技术决策

### 3.2.1 采用前后端分离架构

理由：

React 前端专注桌宠交互、Live2D 渲染和流式消息展示；FastAPI 后端专注数据持久化、AI 调用、认证和服务编排。该结构利于并行开发，也便于后续分别部署前端静态资源和后端 API 服务。

### 3.2.2 使用 SSE 实现 AI 流式输出

理由：

AI 回复可能持续数秒甚至更久，如果等待完整响应，用户会感到卡顿。SSE 可以将后端生成的增量文本逐段推送给前端，降低首屏等待时间，并可以同时传递进度、情绪、完成等结构化事件。

### 3.2.3 使用 manifest 管理 Live2D、背景和音乐资源

理由：

桌宠项目资源种类多，如果硬编码模型路径，后续扩展成本较高。通过 manifest.json 描述模型、背景、音乐、表情动作映射，可以让资源添加更加稳定，前端只负责读取清单并渲染。

### 3.2.4 语音能力采用可替换服务

理由：

ASR 和 TTS 服务在本地、云端和不同模型之间差异较大。项目通过 SC\_ASR\_ENGINE、SC\_WHISPER\_API\_URL、SC\_FUNASR\_API\_URL、SC\_GPT\_SOVITS\_BASE\_URL 等配置项将语音服务外置，便于在开发、演示和部署环境切换。

### 3.2.5 Agent 技能使用本地 SKILL.md

理由：

技能说明以 Markdown 存放在本地目录，开发者可以直接添加或修改技能，而不需要改动主业务代码。Agent 服务读取这些技能后注入模型上下文，使系统具备可扩展的任务能力。

## 3.3 核心代码说明

app/main.py：创建 FastAPI 应用，挂载 API 路由，初始化数据库，提供健康检查，并在前端构建完成后托管静态页面。

app/core/config.py：集中声明项目配置，包括数据库、JWT、AI 模型、语音服务、ASR 引擎、Live2D 资源目录和文件大小限制。

app/services/message\_service.py：项目后端的核心业务服务，负责会话解析、用户消息保存、AI 消息构建、流式调用、Agent 调用、情绪输出和助手消息落库。

app/services/voice\_service.py：封装 ASR 与 TTS 逻辑，支持 Whisper/FunASR 转写和 GPT-SoVITS 合成。

app/services/live2d\_service.py：读取 Live2D 模型 manifest，返回模型列表，读取和保存情绪映射。

app/services/agent\_service.py：负责构建 Agent 模式消息、提供工具定义、执行技能激活、网页获取、受限 shell 和会话摘要。

frontend/shinobu-chat/src/App.tsx：前端主状态容器，管理登录、会话、消息流、Live2D 资源、语音、背景、音乐和桌宠设置。

frontend/shinobu-chat/src/api/sse.ts：解析 SSE 原始事件，将后端事件标准化为前端可使用的 StreamEvent。

frontend/shinobu-chat/src/live2d/Live2DStage.tsx：加载 Cubism Core、创建 Pixi 应用、载入 Live2D 模型，并处理拖拽、缩放、表情动作切换。

frontend/shinobu-chat/src/chat/messageState.ts：处理流式回复期间本地消息和服务端历史消息的合并，避免重复消息或 pending 消息丢失。

## 四、与 Agent 协作总结

### 4.1 协同方式

使用工具：AI 编程 Agent。

使用阶段：需求整理、架构拆分、前后端开发、接口联调、UI 调整、语音功能扩展、测试补充和报告生成。

协作方式：

- 1 用户提出功能方向和当前问题。
- 2 Agent 阅读项目结构和现有代码。
- 3 Agent 将需求拆成后端接口、前端交互、资源配置、测试验证等子任务。
- 4 Agent 根据已有代码风格进行增量实现。
- 5 用户通过运行、截图、日志或 IDE 状态反馈问题。
- 6 Agent 继续定位问题并修复。

协作频率：高频，适合该项目这种多模块、多资源、多交互状态的快速迭代。

### 4.2 典型协同案例

案例：语音交互与 Live2D 情绪联动开发。

- 1 用户提出目标：让 ShinobuChat 不只是文字聊天，还能通过语音输入和语音输出进行交流。
- 2 Agent 分析现有消息流，确认语音功能应接入 Composer、后端 voice route、TTS/ASR service 和消息发送流程。
- 3 Agent 增加后端 /voice/asr、/voice/tts、参考音频配置和情绪识别逻辑。
- 4 Agent 在前端加入录音、语音转文字、回复后合成语音播放等流程。

- 5 系统进一步把情绪结果传递给 Live2D 舞台，实现语音、情绪和角色动作之间的关联。
- 6 最终形成“说话 -> 识别 -> AI 回复 -> 情绪识别 -> 语音播放 -> 角色表情变化”的完整链路。

该案例体现了 Agent 在复杂跨端功能中的优势：它可以同时理解后端服务、前端状态、浏览器 API、外部模型接口和资源配置，并把它们整合为一个可运行流程。

### 4.3 协同经验总结

- 1 先明确数据流，再写代码。语音、AI、Live2D、消息持久化之间存在多段异步流程，必须先确认输入输出边界。
- 2 每个功能都要保留降级路径。例如 Live2D 模型加载失败时显示 fallback，语音服务不可用时返回清晰错误。
- 3 本地配置要样例化。voice\_reference\_audio.example.json 的存在可以降低语音服务配置成本。
- 4 前端流式状态必须单独测试。SSE 中最容易出错的是重复消息、pending 消息丢失和完成事件替换失败。
- 5 Agent 可以加快实现速度，但项目主导者仍需要判断功能取舍、体验质量和演示优先级。

## 五、问题与调整

### 5.1 遇到的问题与解决方案

#### 问题 1：AI 回复存在等待时间

原因：

AI 生成回复需要模型推理时间，长回复或 Agent 模式下等待更明显。

已采取方案：

- 使用 SSE 流式输出。
- 前端逐段渲染 chunk。
- 通过 progress 事件展示 Agent 执行阶段。
- 保存完成消息并替换本地 pending 消息。

后续优化：

- 增加取消生成。
- 增加失败重试。
- 对长回复加入更细粒度的状态提示。

#### 问题 2：中文文案存在编码异常

原因：

部分源文件中的中文字符串已经出现 mojibake，可能来自文件编码、终端编码或历史写入过程不一致。

影响：

- 前端可见按钮、提示、欢迎语可能显示乱码。
- 后端 Agent system prompt、技能关键词、情绪关键词可能无法正确匹配中文。
- 测试中的中文样例可读性下降。

解决方案：

- 统一所有源文件为 UTF-8。
- 修复前端可见中文文案。
- 修复后端 Agent prompt、EmotionService 关键词和 SkillRegistry 关键词。
- 增加一次编码检查，避免再次写入异常文本。

### 问题 3：角色卡接口前后端不完整

原因：

前端 CharacterEditor 已调用 /characters/active，但后端当前 API 路由中尚未注册 characters 模块。

影响：

设置面板打开后，角色编辑区域可能无法加载角色卡。

解决方案：

- 新增 CharacterCard 数据模型或配置文件存储方案。
- 新增 /api/v1/characters/active GET/PUT 接口。
- 将角色卡内容注入普通聊天和 Agent 模式的 system prompt。
- 增加前端错误降级提示。

### 问题 4：接口鉴权仍不完整

原因：

当前登录流程能够生成 JWT，但会话和消息接口多数仍通过 user\_id 参数识别用户，没有统一从 Authorization header 中解析当前用户。

影响：

如果进入正式部署环境，仅依赖前端传入 user\_id 不够安全。

解决方案：

- 新增 `get_current_user` 依赖。
- 前端请求统一携带 Bearer token。
- 后端接口从 token 获取用户 ID。
- 移除或降低外部传入 `user_id` 的信任级别。

## 问题 5：设备授权状态使用内存存储

原因：

当前 `PENDING_DEVICE_AUTH` 是进程内字典，适合 MVP，但不适合多实例或服务重启。

影响：

服务重启会导致授权状态丢失，多进程部署时不同进程之间无法共享设备授权记录。

解决方案：

- 将 pending 授权记录写入数据库或 Redis。
- 增加过期清理。
- 保存授权状态、设备码、用户码、过期时间和审批状态。

## 问题 6：测试覆盖仍偏前端

原因：

当前已有前端测试，但后端服务层、API 路由、语音服务和 Agent 工具还缺少自动化测试。

影响：

后端重构和权限改造时回归风险较高。

解决方案：

- 增加 `AuthService`、`ConversationService`、`MessageService` 的单元测试。
- 对 `/messages` SSE 响应增加集成测试。
- 对 `VoiceService` 外部服务失败、空音频、无 API key 等场景增加测试。
- 使用测试数据库隔离开发数据。

## 5.2 计划调整

提前完成：

- 基础前后端工程搭建。
- 三栏式桌宠聊天界面。

- Live2D 模型加载和情绪映射。
- AI 流式聊天。
- 会话和消息持久化。
- 语音输入与语音输出基础链路。

延后完成：

- 角色卡后端接口。
- JWT 统一鉴权。
- 中文编码彻底修复。
- 后端测试。
- 部署脚本与演示环境。
- 主动提醒和长期偏好系统。

功能无取消，但优先级发生调整：为了保证中期演示效果，项目优先完成可见的桌宠聊天、Live2D、流式对话和语音链路；安全、角色卡、部署和测试将在后续阶段集中收敛。

## 5.3 风险评估更新

已解决风险：

- AI 回复等待期间无反馈的问题。
- 会话刷新后历史丢失的问题。
- Live2D 资源路径硬编码的问题。
- 语音服务难以切换的问题。

当前主要风险：

- 中文编码异常影响演示观感和中文关键词匹配。
- 鉴权未完全闭环，正式部署前必须修复。
- 角色卡接口缺口会影响设置面板完整性。
- 外部 AI、ASR、TTS 服务依赖环境变量和本地服务配置，演示前需要提前检查。

整体风险等级：中等偏低。核心演示链路已经形成，但正式发布前仍需要一次集中稳定性修复。

## 六、后续计划

### 6.1 剩余功能规划

优先级 1：修复中文编码与可见文案。

目标：

- 所有前端按钮、提示、欢迎语恢复正常中文。
- 后端 Agent prompt、技能关键词、情绪关键词恢复正常中文。
- 确保 IDE、终端和文件均以 UTF-8 处理。

优先级 2：补齐角色卡后端接口。

目标：

- 支持读取当前用户角色卡。
- 支持保存语气参数和额外提示词。
- 将角色卡注入 AI 对话上下文。
- 前端 CharacterEditor 可以正常加载和保存。

优先级 3：完成 JWT 鉴权闭环。

目标：

- 前端所有业务请求携带 Authorization header。
- 后端接口通过 token 获取当前用户。
- 会话、消息、角色卡、设备等资源都按当前用户隔离。

优先级 4：增加后端测试与集成测试。

目标：

- 覆盖注册、登录、会话、消息、SSE、语音配置错误等关键路径。
- 使用测试数据库或 mock 服务隔离外部依赖。
- 将前端现有测试纳入常规验证流程。

优先级 5：完善部署与演示。

目标：

- 提供后端启动命令、前端构建命令和环境变量示例。
- 明确 PostgreSQL、GPT-SoVITS、Whisper/FunASR 的启动与配置步骤。
- 准备 3-5 分钟演示脚本。

## 6.2 下阶段重点

未来 2-3 周的重点任务如下：



- 1 完成编码修复和界面中文检查，保证演示观感。
- 2 实现角色卡 API，并让角色设定真正影响 AI 回复。
- 3 将认证从“前端保存 user\_id”升级为“后端解析 token”。
- 4 为消息流、语音接口和 Agent 工具补充后端测试。
- 5 整理部署说明，验证前端 build 后由 FastAPI 托管的完整运行路径。
- 6 优化异常提示，使 AI key、TTS 服务、ASR 服务未配置时能给出明确可理解的错误。

## 6.3 期末目标

- 1 完成功能闭环：

用户登录 -> 进入桌宠工作区 -> 选择角色/背景 -> 文字或语音聊天 -> AI 流式回复 -> 情绪联动 -> 语音播报 -> 会话历史持久化。

- 1 完成智能特性：

聊天模式、Agent 模式、技能调用、长期会话记忆、角色卡设定、情绪识别和主动提醒基础能力可用。

- 1 完成工程稳定性：

前端测试、后端测试、编码修复、鉴权闭环、错误降级和部署说明全部完成。

- 1 完成演示效果：

项目能够在本地或部署环境中稳定运行，演示时可以展示注册登录、Live2D 桌宠、AI 流式聊天、语音输入输出、情绪动作和会话历史。

- 1 达到可扩展状态：

后续可以继续扩展更多角色、背景、音乐、技能、长期记忆、主动提醒和桌面端封装，而不需要重写主架构。

## 七、中期结论

截至目前，ShinobuChat 已经完成从“普通 AI 聊天应用”到“桌宠式 AI 伴侣系统”的主要技术骨架。项目具备清晰的前后端分离结构，后端可以处理用户、设备、会话、消息、AI 流式回复、Agent 技能、Live2D 资源和语音服务；前端已经形成具备辨识度的三栏式桌宠聊天界面，并完成 Live2D、语音、音乐、背景、设置和消息流式渲染等核心体验。

从中期状态看，项目最有价值的成果不是单个接口或页面，而是已经打通了多模态交互链路：用户可以通过文字或语音与 AI 对话，AI 回复可以流式显示、保存到会话历史、触发情绪识别，并进一步联动 Live2D 表情和语音合成。这说明项目核心方向可行，技术架构也具备继续扩展的空间。

后续阶段的重点不再是大规模增加新功能，而是集中补齐完整性和稳定性：修复中文编码、补齐角色卡接口、完善 JWT 鉴权、增加后端测试、整理部署流程。完成这些工作后，ShinobuChat 将具备较完整的课程项目展示价值和继续产品化迭代的基础。